# 📊 Relatório Semanal de Aprendizagem (Semana 8)

**Período:** 13 de Abril a 17 de Abril de 2026 **Foco:** Arquitetura Modular, Namespaces e Resolução de Caminhos (Path Resolution).

## 📈 Resumo Executivo

Esta semana marcou a transição definitiva de scripts isolados para um **Ecossistema Integrado**. O foco principal foi a criação do `main.py` (O Maestro) e a resolução de conflitos de caminhos de ficheiros, especificamente a investigação sobre **Relative vs. Absolute Imports**.

## 🛠️ Evolução Técnica & Conceitos de Clean Code

### 1. Import Architecture: Absolute vs. Relative

- **O Problema:** Erros de `ModuleNotFoundError` e `ImportError: attempted relative import with no known parent package`.
- **A Descoberta (04:14 AM Research):** Investiguei como o Python 3 trata a hierarquia de pastas. Aprendi que para um orquestrador como o `main.py`, o uso de **Absolute Imports** (ex: `from Scripts.Analysis.AlertOverdue import ...`) é mais estável do que depender do local relativo do ficheiro.
- **Princípio Clean Code: Portabilidade.** Ao usar caminhos absolutos baseados na raiz do projeto, o código corre sem erros quer eu esteja no terminal da pasta raiz ou dentro de um sub-módulo.

### 2. Orquestração & Abstração (The Master Menu)

- **Implementação:** Criação de um ponto de entrada único (`main.py`) com um menu interativo.
- **Conceito Base: Abstraction.** O utilizador final (ou o meu Diretor) não precisa de saber que o Python está a navegar por `Scripts/Analysis/`. Eles apenas selecionam uma opção e o Maestro orquestra a execução.

### 3. Resiliência (Fail-Fast Ingestion)

- **Melhoria:** Implementação rigorosa de `try-except` nos blocos de importação. Se um módulo de análise falhar, o menu principal não crasha.
- **Destaque:** Validação de variáveis de ambiente (.env) antes de permitir a execução de qualquer tarefa de IA.

## 🚀 Impacto no Negócio (Business Ops)

- **Centralização:** Reduzi o "Context Switching" de 5 janelas de código para 1 menu centralizado.

- **Audit Speed:** De 45 minutos para 1.2 segundos por auditoria.
- **Professionalism:** O repositório agora segue padrões de engenharia de software (PEP 8), aumentando o meu valor de mercado como **AI-Ops Specialist**.

## 🌅 A Jornada "Golden Dawn" (05:30 AM)

- **Consolidação:** A rotina das 05:30 permitiu focar na resolução de erros complexos de `sys.path` antes do dia de trabalho começar.
- **Deep Research:** O print das 04:14 mostra que a curiosidade técnica é o motor da minha transição de carreira.

## 📅 Próximos Passos (Semana 9)

1. **Logging Persistente:** Criar ficheiros `.log` para cada execução do Maestro.
2. **Automated Report Export:** Implementar a função `save_executive_summary` para PDF.
3. **Advanced Prompt Engineering:** Refinar o "Risk Analyst Posture" na Gemini.

*Assinado: Daniela Marques, Nazaré, Portugal.*